

## 第十四章 网络协议栈与系统

通过本书前序章节的学习，相信读者已经基本掌握了单台计算机操作系统的主要功能和基本原理，如进程、调度、进程间通信（IPC）、内存管理和文件系统等。因为每台机器的计算、存储资源有限，有时需要将不同的计算机进行通信互联，组成可以动态扩展的网络，完成资源共享、作业并行等一系列有趣却又充满挑战的任务。在这一章，我们将介绍一种超越地理位置限制、不受体系结构影响的“超级进程间通信”——网络通信。

本章并不会覆盖计算机网络原理的所有内容，在网络协议栈的介绍上也有所取舍。本章的目的是帮助读者建立计算机网络系统的整体概念，从操作系统的角度理解不同的网络系统设计，以及如何针对不同需求选择满足对应场景的网络设计方案。

### 14.1 网络系统的发展简史

网络系统的发展和计算机自身的发展息息相关。二十世纪 60 年代，分时系统的出现允许多用户通过分时复用来共享一台中央大型器（Mainframe）。彼时的网络模型和现在的云计算场景有些相似，每台终端通过网络线路接入大型机共享计算资源。进入微机时代后，人们将微机同小型机和工作站连接成**局域网**（Local Area Network, LAN）。这一阶段诞生了**分组交换**（Packet Switching）网络——阿帕网（ARPANET），也就是最早的互联网（Internet）。

早期网络有很大局限，不同网络设备使用不同的网络协议，导致局域网内只能使用相同厂商的设备。于是，Vinton Gray Cerf 和 Robert Elliot Kahn 设计了**TCP/IP 网络协议**，解决了异构网络设备之间的计算机互联问题，二人也因此获得了 2004 年度的图灵奖。本章第 14.3 小节将围绕**网络协议栈**展开，帮助读者理解现代互联网工作的基本原理，以及操作系统是如何支持网络通信的（本章第 14.5 小节）。

万维网 (World Wide Web, WWW) 的出现又催生了大量互联网技术, 如搜索引擎、线上购物、视频会议等。移动互联网时代更是诞生了在线支付、短视频等热门网络应用。5G 通信技术又将进一步赋能物联网、边缘计算、自动驾驶等新兴应用场景。新兴网络应用对网络系统的性能有了更高的要求, 特别是处于数据中心内部的网络服务端需要支撑更多并发请求, 同时尽可能减少网络延迟。本章的最后将讨论**数据平面与控制平面分离** (本章第14.6.2小节) 和**智能网卡** (本章第14.6.4小节) 等网络系统的性能优化方案。

## 14.2 一个网络包的生命周期

本节主要知识点

- ❑ 计算机网络分为哪几层？
- ❑ 一个网络包请求是如何发出并最终达到目的地的？

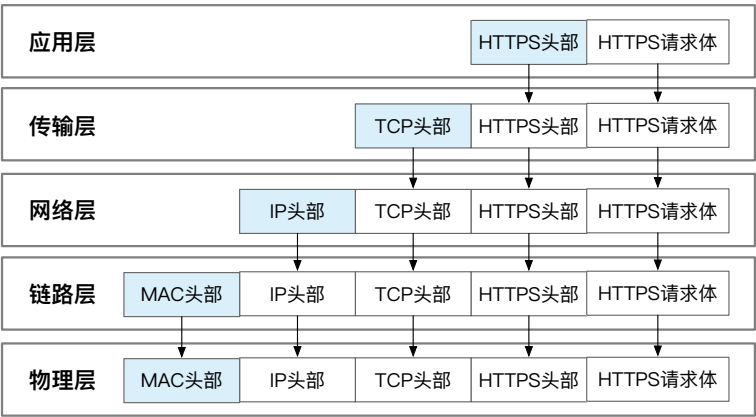


图 14.1: 网络包的发送阶段 (一个网络请求是自上而下传递的, 每到新的一层都会封装一个新的头部)

当你在浏览器的输入框里键入 IPADS 实验室主页的 URL 地址: `https://ipads.se.sjtu.edu.cn/` 时, 浏览器会根据这个地址生成一个数据包, 数据包里表明这是针对 IPADS 主页的 HTTPS 网页请求。如图14.1所示, HTTPS 请求包的发送会自上而下经历协议栈的各个层次, 每经过一层都会增加该层对应的头部信息。增加了所有层次头部的网络包最终会被发送到了互联网上, 由上一节介绍到的路由器、交换机等设备一路传输到对应的服务器上。服务器存

有 IPADS 主页的文件。网页请求的目的就是将这个文件回传到你机器上，由浏览器渲染成可供浏览的多媒体网页。

为了保证对网站的请求一定被响应，浏览器会将 HTTPS 请求包交给 **TCP 协议**。TCP 协议是可靠的传输协议，哪怕请求半路丢失或超时也会自动发起新一轮尝试。既然 HTTPS 数据包使用了 TCP 协议进行传输，操作系统会在 HTTPS 请求包前添加 TCP 头部。这样接收方在解析完整数据包会知悉发送方使用了可靠的 TCP 协议，自己返回时也应使用 TCP 协议，从而保证网页请求的顺利返回。由此可见，TCP 协议主要负责应用层数据的传输，因此我们将 TCP 所在的工作层次称为**传输层**。

这个网页请求仅仅有了 TCP 协议的可靠性传输保证还不够，因为机器还不知道该把这个请求发往何处。为了获悉 IPADS 站点到底运行在哪台服务器上，网络的设计者引入了域名系统（Domain Name System），简称 DNS 服务器。DNS 服务器会负责将人们容易读懂或记忆的网址字符串解析为与之对应的网络 IP 地址。借助 nslookup 命令，我们可以查询 DNS 服务器的域名记录，快速找到 IPADS 网站所在服务器的 IP 地址：

```
# 查询 ipads.se.sjtu.edu.cn 对应的 IP 地址
$ nslookup ipads.se.sjtu.edu.cn

Non-authoritative answer:
Name:   ipads.se.sjtu.edu.cn
Address: 202.120.40.85
```

#### 输出记录 14.1: 用 nslookup 命令查找 IPADS 服务器 IP 地址

知道了 IPADS 网站的 IP 地址是 202.120.40.85，操作系统就能为请求填写收发机器的具体地址了。发送地址即为浏览器所在机器的 IP 地址，用于未来接收返回包裹时使用，接收地址即为 202.120.40.85——IPADS 网站的机器地址。因为 IP 地址就是网络地址，我们将服务于 IP 地址的工作层次称为**网络层**。我们把新添加的信息作为网络层的头部，即 IP 头部。网络层的主要任务就是对 IP 地址进行寻址，即怎么将数据包从发送机器路由到目的机器手中。路由协议也称为 **IP 协议**。我们可以把 IP 协议看成是点对点的通信协议，负责一台设备到另一台设备的网络通信。

借助 IP 头部，这个数据包就标识了收发的具体地址，也就意味着可以将该数据包处于“就绪”状态，可以随时发送到网络上。真实的网络传输还需要再经过一道工序，即对数据包贴上和网络设备的相关信息——MAC 地址。MAC 地址是每块网卡设备出厂时被厂商赋予的身份号码。MAC 地址和 IP 地址都能

起到寻址的作用，MAC 地址可以当做物理地址，而 IP 地址则被认为是逻辑地址，这里的基本原理和内存管理时的虚拟地址和物理地址映射类似。MAC 地址工作的层次称为**链路层**，因此新添加的链路层信息也称为 **MAC 头部**。此时的数据包已经可以发到物理传输介质上进行传输了。这些传输介质构成了**物理层**。物理层直接拷贝了链路层的数据包，不再做任何修改。因此 **MAC 头部**也会反映在物理层的数据包里。

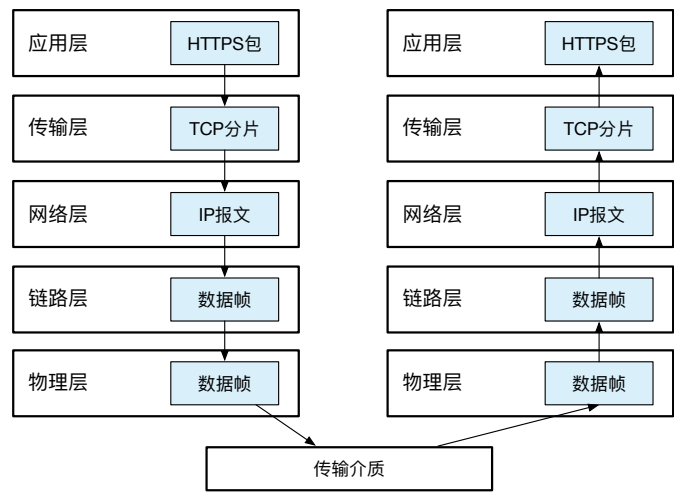


图 14.2: 网络包的跨机器传输过程

数据包到达 IPADS 服务器的网卡后，通常网卡会过滤掉目标 MAC 地址不是自己的网络包。**链路层**的网卡驱动随后把数据包 **MAC 头部**去掉，并送到**网络层**。**网络层**确认这个包的目标地址确实自己的 IP 地址后，大胆地打开包裹，去除 IP 头部，把剩下的报文递交给**传输层**。**传输层**会进一步判断收到的数据包是 TCP 协议还是 UDP 协议。因为 HTTPS 请求使用可靠的 TCP 协议进行封装，所以传输层会使用 TCP 协议进行解析。传输层同时判断是哪个进程拥有了该 TCP 套接字的端口，待一切确认完毕后，这个包会最终会来到套接字之上的**应用层**。应用层很可能是某个 Apache 或 Nginx 的 Web 服务器，负责完成 HTTPS 的解析，并读取文件系统上的 IPADS 网页信息进行返回。最终，你会收到 IPADS 主页的信息，并点击某个按钮再次发起一次网络请求。我们将整个网络包传输的过程用图 14.2完整串联起来。和发包相反，收包是个自底向上的过程，逐层依次解析每一层的头部，并最终得到请求的内容。

## 14.3 网络协议栈

### 本节主要知识点

- “端到端观点” 对网络系统的设计有什么影响？
- 如何定位使用网络通信的具体进程？
- *TCP* 协议和 *UDP* 协议的区别是什么？

在该小节中，我们将重点讨论 *TCP/IP* 网络协议栈。首先我们介绍网络系统设计者的亲身设计经验——端到端观点，随后对上一节中讲的不够具体的协议栈分层内容做进一步展开介绍。

### 14.3.1 协议栈设计经验：端到端观点

网络协议栈的设计经历了较长的发展历史，有一条来自设计者的经验叫“端到端观点”（*End-to-End Argument*）：“通信系统的可靠性，应该依赖于通信两端的应用程序自己来保证。低层的中间件和协议无法做到绝对的可靠；低层对可靠性的支持甚至会损害效率” [14]。注意这个观点并非原理、准则或信条，它是 *TCP/IP* 协议栈亲身设计者的一般性建议。

**如何保证网络传输文件的可靠性？** 我们以论文中的跨机器文件传输为例。假设两台计算机 **A** 和 **B** 之间需要传输一个文件，双方都各自有文件系统和存储介质。将文件从 **A** 传到 **B** 需要借助 **AB** 之间的网络进行通信。错误可能出现在网络传输时的任何阶段，导致传输后的文件完整性遭到破坏（即数据可能发生缺失或某些字节遭到破坏）。

我们将文件的网络传输过程简单分解为三个阶段：

1. 机器 **A** 通过文件系统从存储介质中读取文件，并将文件数据分成适合网络传输的数据包；
2. 数据包通过连接 **AB** 的网络传输到机器 **B** 上；
3. 机器 **B** 收到数据包并组装成文件，最后将文件写入 **B** 的存储介质。

三个阶段从逻辑层面上看非常直接。但在实际过程中，可能会由于各种原因导致文件传输无法正确完成，如存储设备的硬件错误、文件系统的缓冲区溢出、通信链路的丢包或传输时发生的比特错误等。

那么，很自然想到的解决思路是在通信过程中增加**检错纠错**机制：一旦在通信过程中发现数据包有问题，就要求对出错的数据包进行重新发送，使得到达机器 B 的数据包都是正确的。由于连接机器 A 和 B 的网络在通信中起到桥梁的作用，那么是不是可以把数据包的检错纠错任务交给网络呢？

**可靠性到底交给哪一层来保证更合适？** 在第14.2节中介绍了 TCP/IP 协议使用的分层设计，每一层都力求简洁（KISS, Keep It Simple and Stupid）并尽力做好自己的事情。到底什么应该放在底层实现，什么应该放在上层实现，这是设计者须要考虑的现实问题。

那么，用于处理传输错误的**超时重传**机制，是否所有的上层应用都需要它呢？以我们第一小节讨论的打开网页为例，因为用户想要看到页面的内容，因此需要这一机制来保证网页能得到展示。然而，对于音视频业务（如视频会议、线上直播等），用户更加关心的是自己能不能尽快得到听众的回复，对于消息内容的不确定性，听不清的用户可以自主进行再确认。因此，并非所有上层应用都需要**超时重传**机制。如果在底层统一加入超时重传，对于拥堵时段的视频直播则会导致更多的画面延迟，使得用户体验下降。

进一步的，从音视频的实时通信例子可以看出，用户可以自己来保证网络传输的可靠性。同样的，处于最上层的应用程序（即应用层）也可以自己保证可靠性。当**确认-重传**机制放在应用层时，底层的网络核心部分可以更加专注于数据包的收发，将网络传输速率最大化。这一设计也使网络的功能变得更加简单纯粹，尽可能地降低了网络的复杂性。由此可见，端到端观点可以被形象概括为**笨拙的网络，聪明的终端**（Dumb Network, Smart Endpoints）。

端到端观点的提出者最后将该观点总结为系统设计的奥卡姆剃刀法则（Occam's Razor）：如无必要，毋增实体。如果底层的服务已经超出其核心逻辑，那就应该在更高层去实现这个服务。网络系统的端到端观点和操作系统中的外核（Exokernel）设计思路其实非常相似：当系统无法保证某个服务可以惠及所有上层应用时，那么就可以考虑将该服务交给处于端侧的应用层自己去实现。换言之，当底层无法给出足够通用的方案时，就应该缩小边界，力求简单。

### 14.3.2 TCP/IP 网络协议栈

为了计算机网络体系结构及网络协议的标准化，国际标准化组织专门制定了**开放系统互联参考模型**（Open System Interconnection Reference Model, 简称 OSI 模型）。由于 OSI 模型将网络分成七层，因此也被称之为 OSI 七层

模型。OSI 七层模型明确定义了各层的功能和层间的接口，有利于网络子系统的模块化开发和功能扩展。然而，在计算机网络的发展过程中，OSI 模型并未得到普及。美国国防部创建的 TCP/IP 五层模型在计算机网络发展过程中占据了主导地位，成为了既定的事实标准。表14.1展示了 OSI 七层模型和 TCP/IP 五层模型的对应关系。接下来，我们从 TCP/IP 的底层依次向上，对每一层的协议设计和作用进行展开。

表 14.1: 计算机网络体系结构层级

| OSI   | TCP/IP | 作用           | 主要协议                   |
|-------|--------|--------------|------------------------|
| 应用层   | 应用层    | 文件服务、电子邮件等应用 | DNS、HTTP、FTP 等         |
| 表示层   | 应用层    | 格式、压缩、加密     | 无对应                    |
| 会话层   | 应用层    | 建立或解除通信      | 无对应                    |
| 传输层   | 传输层    | 进程到进程的通信     | TCP、UDP                |
| 网络层   | 网络层    | 主机到主机的通信     | IP、ICMP、IGMP、BGP 等     |
| 数据链路层 | 链路层    | 构建数据帧传输的链路   | ARP、PPP、MTU、SLIP 等     |
| 物理层   | 物理层    | 提供信息传输的物理介质  | IEEE802.2、IEEE802.11 等 |

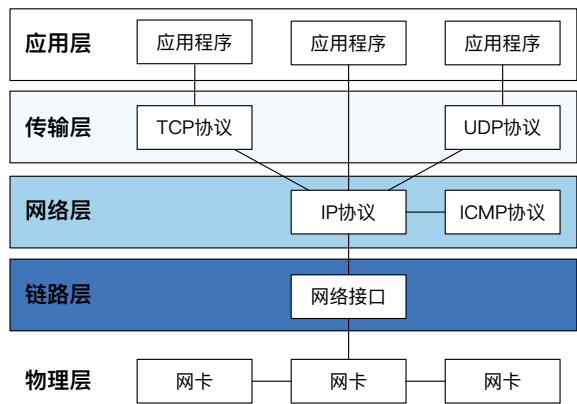


图 14.3: TCP/IP 协议栈的具体分层及其部分协议

物理层和链路层

TCP/IP 模型中的物理层是负责将不同计算机连接起来的物理介质，例如 WiFi 信号中的电磁波、网线中的光纤材料等。依靠这些物理传输介质，信息才能在不同计算机之间传输和扩散。

链路层的主体是以太网网卡（Network Interface Controller，NIC），每张网卡在出厂时都分配了唯一的 MAC 地址。从网卡发出去的每个数据帧都会在其数据帧头部的源 MAC 地址处填上自己的 MAC 地址。

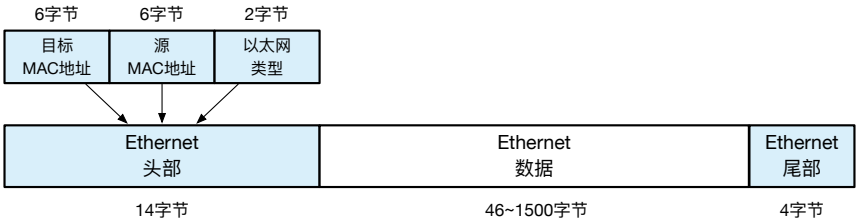


图 14.4: 数据帧的格式

**数据帧：**网络通信并不是将内存上的数据根据其内存字节顺序依次发给对方，相反，网络设计者将 01 比特构成的字节流进行了有效分组，每个分组称之为一个**数据帧（Frame）**，或称为**MAC 帧**。如图14.4所示，数据帧分为头部、载荷（Payload）和尾部。头部包含了双方的地址，即目标 MAC 地址和源 MAC 地址，以及载荷（Payload）的具体类型。载荷也就是数据帧的数据部分，如 IP 数据包。数据部分不能无限扩容，以太网协议规定，数据部分最多只能到 1500 个字节，一旦超过就必须进行分帧，即分成多个数据帧进行传输。数据帧的尾部用于检验数据帧在传输过程是否出错，通常尾部会被网卡设备自动裁掉，网卡驱动代码读取不到这个尾部。如果检验出现问题，网卡设备直接将数据帧丢弃。

在数据帧的收发阶段，每台网络设备都会仔细“侦听”每一个数据帧，查看其目标 MAC 地址是否是自己，是的话则接纳这一帧。对于接纳的数据帧，网卡首先通过尾部的数据检查数据帧是否完整，若完好无损，则通过中断的形式通知 CPU 上的网卡驱动程序来处理。网卡驱动会对数据帧的头部进行剥离，然后根据头部的类型决定应该交给哪一个上层协议进行处理。

网络字节序 vs. 机器字节序

机器字节序指的是大小端模式：不同的处理器使用不同的大小端模式，整数在内存中使用相反的比特保存顺序。网络字节序使用大端字节序：TCP/IP 协议要求其头部中所有的二进制整数使用大端字节序进行网络传输，因此称为网络字节序。

## 网络层

**IP 协议：**相比于链路层的 MAC 地址，网络层的 IP 协议又引入了 IP 地址，用于不同机器在互联网上的点对点通信。目前的 IP 协议包括了 IPv4 和 IPv6 两个版本。后者相对于前者提供了更大的 IP 地址空间，同时在安全性和性能上做了改进。因为 IPv6 的组网还没有完成，我们这里仍旧以常用的 IPv4 为例。IPv4 使用 32 位地址，地址分为两部分，前面表示网络号，后面表示在局域网中的主机号。例如 192.168.0.1 的前 24 位就是网络号，后 8 位就是主机号。由于 IPv4 的地址面临短缺的压力，为了提高 IP 地址的利用率，IP 网络又可以划分为更小的子网（Subnet）。如果两个 IP 地址在同一个子网内，则网络号一定相同。从这里可以看出，MAC 地址描述的是网络设备的绝对地址（物理地址），而 IP 地址则描述了机器间的网络拓扑关系（逻辑地址），比如两台机器是否处于同一子网。

相比于 MAC 地址，引入 IP 地址有什么好处呢？假设某块网卡发生了故障，导致某台主机无法上网，此时只需为该主机更换网卡即可解决问题。此时 MAC 地址虽然发生了修改，但原先使用的 IP 地址却可以直接复用，只须在子网内部广播更新后的映射。

### 小知识：

**网络地址转换（NAT）**可以将来自私网数据包的 IP 地址转换为公网 IP 地址。所谓私网 IP，是指该 IP 只在某子网内可见，公网 IP 则意味着一个互联网全局可达的地址。由于 IPv4 公网地址资源相对稀缺，NAT 技术可以节省 IP 地址资源，允许多台私网主机共享一公网 IP 地址。此外，NAT 也有利于保护私网主机免受外部网络攻击，因为 NAT 隐藏了私网内部结构和私网主机的真实 IP 地址。

**IP 数据包：**用 IP 协议封装的数据称为 IP 数据包（IP Packet），简称数据包。数据包是网络协议栈主要的处理对象。理论上数据包的长度可以不受限制，但在具体实现中，如 IPv4 的头部使用了 2 个字节描述数据包的大小，因此数据包最长不能超过 64KB。然而链路层的数据帧一次却只能传输最长 1500 字节的载荷。为了解决这一问题，就不得不对数据包进行分片（Fragmentation）。这些分片通过不同的数据帧发往目标机器后，会在目标机器的网络层进行重新组装。对于数据包而言，其目标 IP 地址和源 IP 地址可能会跨越多个子网，因此网络层设计了专门的路由协议，让数据包能在不同子网内高效地转发。

**ICMP 协议：**为了检查网络的通达性，网络层还引入专门的心跳协议——ICMP 协议，用于判断对方机器是否在线。我们可以用 ping 命令向 IPADS

服务器发送心跳包。`ping`命令将对方发回的 **ICMP** 数据包（即回音，`echo`）按序显示，并用`icmp_seq`字段表示。`icmp_seq`从 0 开始计数，表示这是第几次`ping`对方从对方收到的回音。如果网络发生拥堵的情况，则会有部分`icmp_seq`没有显示，表示这些序号的 **ICMP** 数据包没有收到。

```
$ ping ipads.se.sjtu.edu.cn
PING ipads.se.sjtu.edu.cn (202.120.40.85): 56 data bytes
64 bytes from 202.120.40.85: icmp_seq=0 ttl=41 time=61.509 ms
64 bytes from 202.120.40.85: icmp_seq=1 ttl=41 time=56.554 ms
64 bytes from 202.120.40.85: icmp_seq=2 ttl=41 time=59.540 ms
```

输出记录 14.2: 用 `ping` 命令向 IPADS 服务器发送心跳包

传输层

尽管网络层引入了 **IP** 地址标识出了一台机器在互联网空间中的具体位置，但是这依然不能满足进程间跨机器网络通信的需求。假设某台机器上同时运行着多个应用程序，当数据包发送至目标机器后，如何确定这个包到底是发给哪个应用程序的呢？

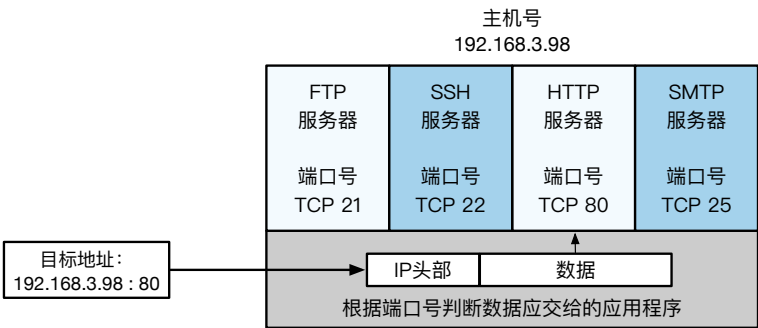


图 14.5: 网络协议栈根据端口号识别应用进程

**端口号**：传输层也提供了类似 **IP** 地址的定位机制——端口号，用于定位一台机器上不同进程。传输层协议利用这些端口号识别本机中正在进行通信的应用程序，并进行准确地数据传输。因此，可以把端口号理解为网络程序在一台机器上的准确地址。常见的端口号确认方法有两种：

- 1. **静态分配法**：给每个网络应用分配固定端口号。例如 **HTTP**、**FTP**、**SSH** 等广为使用的应用层协议所使用的默认端口号就是固定分配的。当然也

可以由系统管理员在网络应用启动时手动进行分配。

2. **动态分配法**：服务器端有必要确定用于监听的端口号，但是使用服务的客户端则无需确定端口号。这种情况下，客户端应用程序的端口号全权交给操作系统进行分配。

**TCP 协议**：TCP 是面向连接的、可靠的字节流协议。字节流（Stream）是指不间断的数据结构。由于 IP 协议传输数据时本身是不可靠的，可能存在丢包、乱序到达等问题，TCP 使用确认应答和超时重传的机制来提供可靠传输。所谓确认应答，是指接收方收到 TCP 包后向发送方回应确认（Acknowledge）信息，发送方收到确认信息后再发送后续 TCP 包。为了避免每次发包都需要对方确认，TCP 设计了可动态调整的滑动窗口（Window），容纳多个未确认的 TCP 包。所谓超时重传，则是指发送方为每个 TCP 包设置一个定时器，如果在规定时间内没有收到接收方的确认应答，则认为该数据包丢失，发送方会选择重新发送该数据包并再次设置定时器，直至确认发送成功为止。

**UDP 协议**：并非所有的网络通信场景都需要 TCP 的可靠连接服务。相比于 TCP 协议，UDP 协议具有如下的特点：首先，UDP 是无连接的，即通信双方无需建立虚拟通信链路即可进行通信；其次，UDP 没有流量控制和拥塞控制机制，因此它能以稳定的速率发送数据包，在丢包率较低的情况下能够较好地支持恒定的视频分辨率；最后，UDP 首部较小，通信时数据报的载荷比（有效载荷大小与整个数据报的比）更高，因此通信效率更高。当然，这些好处是在牺牲可靠性的前提下达到的。传输途中出现丢包，UDP 也不负责重发，甚至当包的到达顺序出现乱序时也没有纠正的功能。使用 UDP 协议的应用程序需要根据自己的需要，选择性地实现一定的流量控制和拥塞控制机制（请回顾第14.3.1小节的“端到端观点”）。

## 14.4 网络编程模型

### 本节主要知识点

- 如何使用套接字接口建立网络连接？
- 如何解决服务端网络并发请求的性能问题？

14.4.1 套接字编程

在本小节中，我们将介绍应用程序和网络协议栈传输层之间交互的接口——套接字。套接字是进程间通信协议的一种抽象，它为应用程序提供标准的API，从而支持远程机器间的TCP/IP通信协议。对于TCP/IP协议，套接字使用IP地址和端口号的组合来确定网络上某台机器中的一个进程。操作系统通过套接字封装了网络层和传输层通信的具体细节，使进程通过简单地读写套接字就能实现跨网络的进程通信。

套接字编程使用客户端-服务器（Client/Server，简称C/S）模型。客户端要访问服务器时需主动发出请求，服务器接收到请求后再提供相应服务。一个套接字对象就代表着通信双方的某一方。进程通过套接字对象来管理通信过程，如建立连接、关闭连接，也通过套接字对象进行数据收发。

常见的套接字连接包括了TCP和UDP两种连接。此外，套接字还允许应用程序绕过传输层直接和网络层打交道，即提供原始套接字（Raw Socket），本书这里不做介绍。接下来我们对套接字的使用进行介绍，以帮助读者了解网络连接请求是如何建立的。

套接字编程

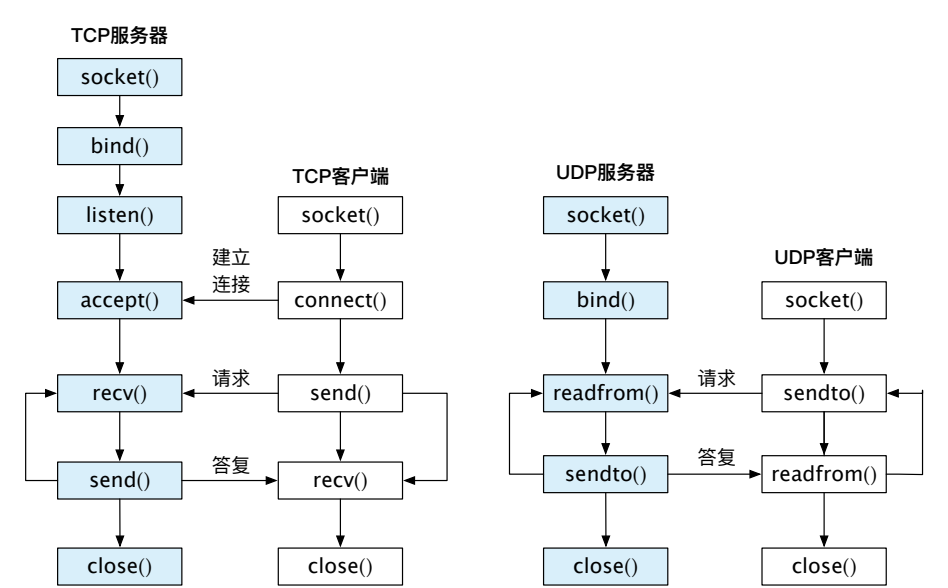


图 14.6: TCP 和 UDP 网络编程状态图

**TCP 服务端步骤：**

1. 创建套接字 `fd1` (`socket`)，通过 `SOCK_STREAM` 将套接字声明为 TCP；
2. 将 `fd1` 绑定 (`bind`) 到特定的 IP 地址和端口；
3. 将套接字设为监听模式 (`listen`)，随时准备接收客户端请求；
4. 当请求到来后，接受 (`accept`) 连接请求，返回一个新的对应于此次连接的套接字 `fd2`；
5. 接收 `fd2` (`accept`)，然后和客户端使用 `send/recv` 进行通信；
6. 客户端断开连接，关闭 `fd2`，等待另一个客户端请求；
7. 服务器准备离开，关闭套接字 `fd1` (`close`)。

**TCP 客户端步骤：**

1. 创建套接字，通过 `SOCK_STREAM` 将套接字声明为使用 TCP 协议；
2. 向 TCP 服务端发出连接 (`connect`) 请求，请求中包含 TCP 服务端的 IP 地址和端口号；
3. 和服务端使用 `send/recv` 进行通信；
4. 通信结束，关闭套接字 (`close`)。

TCP 和 UDP 建立连接的过程请参考图14.6。二者最大的区别是 UDP 并不建立连接，UDP 客户端可以直接向服务端发送数据进行通信。

**小知识：**

在 UNIX 哲学中，一切都是文件 (“Everything is a File”)。显然套接字也不例外。TCP 使用的 `send()`、`recv()` 系统调用和 UDP 使用的 `sendto()`、`recvfrom()` 数据包收发系统调用都可以换做 `write()` 和 `read()`。

**14.4.2 I/O 多路复用 **

在网络编程中，应用程序很有可能建立多个连接并同时与这些连接保持通信，例如常见的 Web 服务器和数据库服务器等。那我们应该如何处理这种场景下的网络通信呢？

如果只使用 `socket()` 的话，可能存在两种相对简单的应对方法：

- 服务端为每个连接都建立一个线程（或进程），从而能并发地处理所有连接。当连接数量较大时，大量线程的建立和维护会占用较多系统资源。

- 服务端将套接字调整为非阻塞，即客户端无信息到达时依然可以返回并继续服务端逻辑。使用非阻塞 I/O 时服务端需要依次遍历所有连接，检查到资源准备就绪的连接就立即进行服务。缺点在于需要耗费处理器资源不断遍历各个连接。

由此可见，使用基本套接字接口处理并发 I/O 操作（即 `read`、`write`）时，通常很难避免较高的系统资源消耗。于是操作系统提出了 **I/O 多路复用**（I/O Multiplexing）的机制来处理这种情况。它的基本出发点是为 I/O 操作增加新的通知机制，允许用户态应用程序首先将等待的多个 `socket` 提前注册，然后统统阻塞，由操作系统异步地通知用户态进行收发相关的逻辑。当注册好的 `socket` 存在可被处理的资源时，返回用户态进行正常的 `socket` 操作。值得注意的是，I/O 多路复用方法不仅可以处理并发的网络 I/O 操作，还能够处理进程间通信、管道等场景，此处我们仅针对网络场景进行讨论。

常见的 I/O 多路复用接口有 `select`、`poll`、`epoll` 和 `kqueue` 等。我们以 Linux 的 `poll` 接口为例介绍此类接口的使用方式。代码片段 14.1 展示了如何使用 `poll` 接口同时等待两个 `socket` 并执行 `recv` 操作。首先需要将所有等待操作的 `fd` 存储至数组 `fds`，并指定要等待的类型为有数据可读（`POLLIN`），类似的还有可写（`POLLOUT`）、出现异常（`POLLERR`）等。然后调用 `poll` 同时等待两个 `socket` 的资源可读，当等待超时或者某些 `fd` 有数据可读时，该调用会返回，即等待结束。对于 `socket` 来说，所谓**有数据可读**是指缓存中存在大于等于 `SO_RCVLOWAT` 个字节的数据，或者对方关闭连接导致读出 `EOF` 等情况。当 `poll` 返回后，应用根据各个 `fd` 的 `revents` 判断该 `fd` 产生了何种事件，同时做出响应处理。

---

```
1 int timeout_ms = 500;
2 int i, accept_fd = 0;
3 struct pollfd fds[2];
4
5 // 存储准备进行操作的 fd 至 fds
6 for (i = 0; i < 2; i++) {
7     accept_fd = accept(server_fd, (struct sockaddr
8         ↪ *)&ac, &socklen);
9     fds[i].fd = accept_fd;
10    // 只等待存在数据可读取的事件
11    fds[i].events = POLLIN;
12 }
13 while (true) {
14     // 同时等待 fds 中的 2 个套接字是否有数据可读, 设置超时
15     ↪ 500ms
16     ret = poll(fds, 2, timeout_ms);
17     if (ret == 0) {
18         printf("timeout\n");
19         continue;
20     }
21     for (i = 0; i < 2; i++) {
22         // 遍历 fds 判断其中是否有数据可读
23         if ((fds[i].revents & POLLIN) != 0) {
24             ret = recv(fds[i].fd, buf, len, 0);
25             // 返回值为 0 表明该 socket 已被远端关闭
26             if (ret == 0) {
27                 close(fds[i].fd);
28                 fds[i].fd = -1;
29             }
30         }
31     }
32 }
```

---

代码片段 14.1: 使用 poll 多路复用接口代码示例

## 14.5 操作系统的网络实现

TCP/IP 协议栈仅仅规定了通信的规范, 只要满足这套规范, 任何节点都可以基于 TCP/IP 进行通信。因此各家有各家的协议栈实现, 有的实现在内核态, 有的实现在用户态, 还有一些实现针对高性能做了优化, 不一而足。本节我们将结合具体实现, 从系统开发者的角度分析不同的网络实现。

### 14.5.1 案例分析：Linux 内核协议栈

#### 本节主要知识点

- ❑ Linux 的网络收发过程是怎样的？
- ❑ Linux 如何实现内核态网络包处理的零拷贝？
- ❑ Linux 内核如何解决不同网络设备的统一管理？
- ❑ NAPI 模式是如何解决高频数据包处理的？

Linux 属于宏内核设计，网络协议栈和网卡驱动都运行在内核态。下面我们介绍 Linux 网络子系统为完成网络功能所提供的主要抽象和加速技术，包括用于管理和跟踪内核数据包的 `sk_buff` 抽象，统一不同网卡设备类型的 `net_device` 抽象，以及用于网络加速的 NAPI 模式。最后我们概述 Linux 网络收发的基本过程。

```
1 struct sk_buff {  
2     union {  
3         struct {  
4             struct sk_buff *next;  
5             struct sk_buff *prev;  
6             ...  
7         };  
8         struct rb_node rbnode; // 红黑树  
9     };  
10    unsigned int len, data_len;  
11    __u16 transport_header;  
12    __u16 network_header;  
13    __u16 mac_header;  
14    sk_buff_data_t tail;  
15    sk_buff_data_t end;  
16    unsigned char *head, *data;  
17    ...  
18 };
```

代码片段 14.2: `sk_buff` 结构的定义

**sk\_buff** Linux 使用一个统一的数据结构 `sk_buff` 来管理内核数据包。`sk_buff` 是 `socket buffer` 的简称，也被缩写为 `skb`。代码片段 14.2 展示了 `sk_buff` 的具体定义。作为 Linux 网络协议栈的核心数据结构，协议栈根据 `sk_buff` 就能获得数据包的所有相关信息，包括数据包的内存地址、数据包大小等。例如，同一数据包的不同分片通过 `sk_buff` 的 `prev` 和 `next` 成员组成

双向链表，协议栈可以使用这个双向链表把各个分片合并成完整的数据包。为了提升效率，Linux 使用红黑树来代替这个双向链表。

值得注意的是，`sk_buff`结构体本身并不存储数据包内容，它通过多个指针指向真正的数据包内存空间。根据当前协议栈的层次，`data`指向缓冲区里保存的数据包的首地址，`head`指向当前协议层所要处理的头部首地址，`tail`指向数据包的尾地址，`end`指向包含数据包的内存块的尾地址。当数据包在不同的层次之间来回处理时，协议栈通过移动`data`指针来处理不同协议层对应的头部。因此，协议栈通过`sk_buff`里的指针成员就能完成数据包的拆包和封装操作，从而避免协议栈中每一层的处理都需要执行内存拷贝操作。因为在各层间交换数据不必拷贝数据包，`sk_buff`显著提升了内核处理网络包的速度。

---

```
1 struct net_device {
2     /* 硬件相关信息 */
3     char   name[IFNAMSIZ];    //设备名，如 eth0
4     int    irq;               //设备中断号
5
6     /* 统计相关信息，包括收发的数据包数目、字节数、丢包数等 */
7     struct net_device_stats stats;
8
9     /* 回调函数，包括设备的注册及卸载、数据包的收发等 */
10    const struct net_device_ops *netdev_ops;
11    ...
12 };
```

---

代码片段 14.3: `net_device` 结构的定义

**net\_device** Linux 将网卡设备统一抽象为`net_device`结构体，使得网络协议栈能够通过操作`net_device`结构体和实际网卡设备进行交互，同时屏蔽硬件设备的差异性和驱动程序的实现细节。`net_device`结构体的定义如代码片段14.3所示。这个结构里包含了网卡的所有信息，包括设备名、中断号等硬件相关信息、网络包的统计信息以及网卡驱动的收发回调函数等接口信息。

借助`net_device`结构体，网络协议栈无需关注网卡设备的具体型号，而是使用标准的接口来收发数据包，如使用函数`dev_queue_xmit()`来发送数据包，使用函数`netif_receive_skb()`来接收数据包。相对应地，设备驱动层无需关注协议栈的具体协议，只需实现`net_device`结构体要求的回调函数，就能获取遵循协议栈封装的数据包并完成收发操作。

**New API (NAPI)** NAPI 是 Linux 用于提升网络包接收速度的处理机制。对网卡数据包的处理上通常有中断和轮询两种方式。中断方式响应及时，如果数据量较小，不会占用太多 CPU 周期。但是当数据包频频到来时，每次都触发中断则会消耗不少的 CPU 周期。轮询方式更适合处理高负载的场合，缺点是网络负载较低时轮询会一直占用 CPU，导致无意义的空转。

开启 NAPI 模式后，当网卡中断发生时，中断处理函数首先会关闭网卡中断，即不再使用中断模式响应网卡数据包，同时确认该中断以允许后续数据包到来，随后立即调度软中断（第十章介绍的NET\_TX和NET\_RX两种软中断）。软中断会调用轮询函数，轮询函数会不断处理在当前网卡中累积的数据包。为了避免单次轮询的时间过长，NAPI 在软中断中分配了单次处理的最大配额以及最长时间。当配额用满或者轮询超时都会结束软中断以退出轮询。软中断结束前会再度打开网卡中断，当下一次网络包到来时会再次触发中断并进行新一轮的轮询。NAPI 的做法有效避免了每次接收数据包时都发生中断，同时也减少了长期轮询导致的浪费 CPU 周期的问题。

**Linux 网络包收发过程概述** Linux 的收包过程主要可以分为三个阶段：

1. 网卡收到数据包后，通过 DMA 将数据拷贝到内核驱动事先分配好的接收队列（RX Ring），随后产生硬中断，触发图14.7中的netif\_rx()中断处理函数；
2. 网卡驱动程序在软中断处理函数内net\_rx\_action()为数据包申请sk\_buff缓冲区对象，同时将数据从接收队列拷贝至sk\_buff对象；
3. 驱动层将sk\_buff上抛给内核协议栈，由协议栈负责完成协议解析处理并根据传输层信息查找到套接字对象。拥有该套接字对象的用户态进程被最终唤醒（图14.7中的wake\_up\_interruptible() 函数），在用户态进程请求读网络数据时把数据从位于内核空间的sk\_buff对象拷贝到用户内存。

类似地，Linux 的发包过程也能分成三个阶段：

1. 用户态应用程序执行套接字操作发送数据，操作系统内核为要发送的数据申请sk\_buff对象，并将数据从用户空间拷贝至内核的sk\_buff中。
2. Linux 协议栈根据sk\_buff对象对数据包进行协议头封装处理，并将数据包拷贝到网卡驱动事先分配好的发送队列（TX Ring）；
3. 网卡通过 DMA 将发送队列上封装好的数据包拷贝并发出。

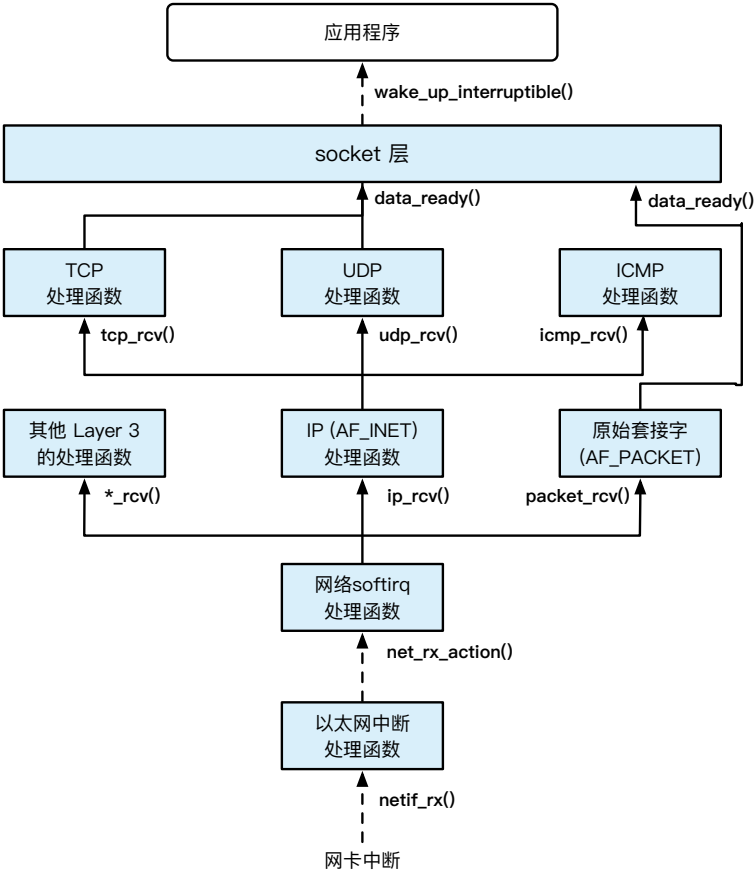


图 14.7: **Linux 网络数据包接收处理过程** [3]（整体数据包的传递使用 `sk_buff` 数据结构。其中虚线表示的是异步过程，实线表示的是控制流直接跳转的过程）

14.5.2 案例分析：ChChore 用户态协议栈

本节主要知识点

- ChCore 的网络中断处理流程是什么样的？
- ChCore 是如何实现用户态网络的？

ChCore 采用微内核设计，因此不同于 Linux 宏内核，ChCore 的网络协议栈以用户态服务进程的形式存在。此外，ChCore 将容易包含 bugs 的驱动程序放置于用户态进行隔离，因此网卡驱动本身也属于用户态进程。对于用户

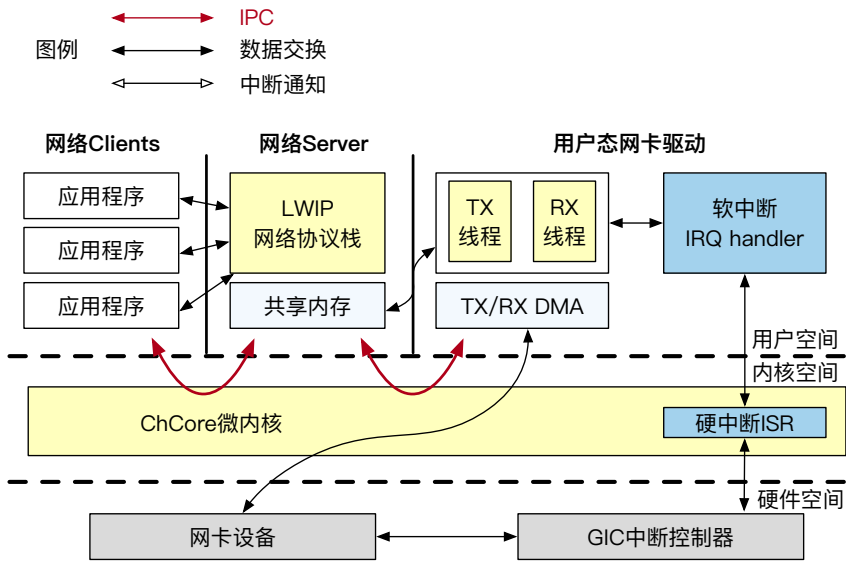


图 14.8: ChCore 的网络设计。网卡驱动、网络协议栈均以用户态进程存在，之间通过 IPC 进行通信。网卡中断被转发至用户态软中断处理程序处理。用户态网卡驱动和网卡设备通过映射至用户态的 DMA 内存区域进行交互

态驱动程序的好处，本书第十章节的相关部分做了详尽的介绍。ChCore 整体网络架构如图14.8所示。

ChCore 的当前实现移植了 lwIP 协议栈和用户态驱动代码。如下我们介绍 lwIP 协议栈在 ChCore 中的移植设计，和 ChCore 的网络中断处理。

lwIP 协议栈

lwIP 是轻量级 TCP/IP 协议（Light Weight TCP/IP）的简称 [15]。它是由瑞典计算机科学院的 Adam Dunkels 开发，无论是否有操作系统的支持均可以运行。lwIP 将所有协议驻留在同一个进程中，以便独立于操作系统核心之外。应用程序使用 lwIP 协议栈可以采用两种方法：一种是函数调用，这适用于应用程序直接链接 lwIP 库并运行在同一个进程的情况；另一种是将 lwIP 和应用程序分别运行在不同的进程地址空间内，然后使用 IPC 的方式获取 lwIP 对协议栈处理的支持。ChCore 使用的是后者。

为了方便移植，lwIP 提供了专门的操作系统抽象层（lwIPOpts.h 和 arch/sys\_arch.h 头文件）。在实际移植时，我们利用 ChCore 所提供的内存管理、进程间通信和同步等系统服务来实现操作系统抽象层。此外，lwIP 使用了

pbuf 来管理数据包，其具体数据结构如代码片段14.4所示。pbuf 用next将多个分片用链表的方式进行管理，同时提供了数据包的基本信息。此外，pbuf 还专门实现了数据包在协议栈处理的零拷贝，其基本原理类似于 Linux 的 skb，这里不再展开。ChCore 的用户态网卡驱动在封装数据包时将数据包封装为 pbuf 数据结构。

ChCore 上的网络处理至少包含了 3 个进程：一个是网络应用程序进程，一个是 lwIP 协议栈进程，还有一个是负责同底层硬件交互的驱动进程，其中驱动进程包含了用户态的中断处理线程。

```
1 struct pbuf {  
2     struct pbuf *next;  
3     void *payload;  
4     u16_t tot_len;  
5     u16_t len;  
6     u8_t /*pbuf_type*/ type;  
7     u8_t flags;  
8     u16_t ref;  
9 };
```

代码片段 14.4: pbuf 结构体的定义

#### 小知识:

在网络协议栈的实现代码中，常常会看到不少 `__attribute__((packed))` 的字样。通常 CPU 都是按照特定字长访问物理内存的，比如在 64 位的机器上，处理器会每次从 64 整数倍的内存地址读取数据，我们称这种访问模式为“内存对齐”。

内存对齐通常是由编译器来完成的，当协议头部封装成 struct 结构体时，编译器默认会根据目标平台的字长将结构体内部成员进行“对齐”，内存大小比对齐字节少的变量会被添加一些字节，这个过程称为“填充”（Padding）。但是网络包可能传输到异构平台上，不一致的填充将导致网络包无法解析。`__attribute__((packed))` 负责在声明结构体时，显式告诉编译器不要填充任何字节。

---

```
1 #define PACK_STRUCT_STRUCT __attribute__((packed))
2 #define PACK_STRUCT_FIELD(x) x
3 #define PACK_STRUCT_FLD_8(x) PACK_STRUCT_FIELD(x)
4 #define PACK_STRUCT_FLD_S(x) PACK_STRUCT_FIELD(x)
5
6 struct ip_hdr {
7     ...
8     PACK_STRUCT_FLD_8(u8_t _proto);
9     PACK_STRUCT_FIELD(u16_t _chksum);
10    PACK_STRUCT_FLD_S(ip4_addr_p_t src);
11    PACK_STRUCT_FLD_S(ip4_addr_p_t dest);
12 } PACK_STRUCT_STRUCT;
```

---

代码片段 14.5: lwIP 中 ipv4 头部的定义（结构体声明使用了 packed 属性）

**ChCore 的网络中断处理：** 在 ChCore 微内核系统中，同网卡交互的网络驱动进程注册了用户态的中断处理线程（IRQ Handler）。该线程默认处于阻塞状态，当网络中断到来时，该线程会被唤醒，同时去访问网卡 DMA 后的环形队列，调用网卡 I/O 处理函数对数据帧进行收发处理。

当线程接收到数据帧后，一种不合理的实现是在软中断上下文中直接调用 IPC 触发 lwIP 的 RX 回调函数，把数据帧发给协议栈进行解析。为什么这么做会出现问题呢？因为相比于接收数据帧，对数据帧的解析是一个更为漫长的过程。试想一下，如果在 lwIP 处理 RX 回调过程中再次发生中断，由于此时数据帧还没有处理完，而网卡 RX 中断还处于屏蔽状态，那么新的中断处理时机将被大大推迟。此时每个中断的处理时间都包含了协议栈对数据帧的解析时间，导致新来的中断得不到及时处理，数据帧处理的实时性会大打折扣。

一个恰当的做法是引入异步 I/O，在协议栈进程中实现为一个专门的 RX 线程，然后将中断处理线程收到的帧通过非阻塞 IPC 的方式传给它。此时中断处理线程只负责将数据从 DMA 缓冲区拷贝给 RX 线程，同时及时确认网卡中断（EOI）。RX 线程可以采用轮询或 NAPI 的方式及时处理新进来的数据帧。这个做法类似于 Linux 中断处理的上下半部设计，保证了处于上半部的中断处理线程的工作量足够少，从而提高中断处理的实时性。

## 14.6 网络系统的性能优化

### 本节主要知识点

- 常用的网络性能优化方案有哪些？
- 如何理解网络系统的“控制平面与数据平面分离”？
- *Intel DPDK* 是如何解决传统网络的性能瓶颈的？
- 智能网卡分为哪几类？引入智能网卡能为数据中心解决哪些问题？

当前网络系统面临着来自底层硬件和上层应用两方面的挑战：

1. **硬件层面**：随着摩尔定律的失效，通用处理器的性能提升逐渐放缓，而网络设备的带宽则有了明显提升。如今处理器的频率提升远远跟不上以太网网卡的提升速度（以太网卡的速度已经从最开始的 10Mbps 提升到了现在的 100Gbps）。为此，人们想出了各种网络加速方式，来降低 CPU 处理单个网络包的处理时间。
2. **应用层面**：大数据与机器学习等领域对算力的需求与日俱增，这些计算通常需要计算节点间进行大量通信。传统数据中心的基础设施主要采用通用处理器，网络请求的处理消耗了大量算力，使得留给应用逻辑的 CPU 资源和内存资源变得紧张。于是人们实现了各种专用计算芯片，包括面向机器学习的 TPU、NPU，以及专为网络服务的智能网卡（SmartNICs）。

下文将首先介绍常见的网络加速方案，其中不少方案已在数据中心进行了大规模实践。接着介绍数据平面与控制平面分离的网络系统设计，以及其中较为经典的 *Intel DPDK* 方案。最后讨论智能网卡等新兴的网络相关技术。

### 14.6.1 常用的网络加速方案

**Remote Direct Memory Access (RDMA)**：传统 DMA 技术允许本地网卡访问本地内存，而 RDMA 技术允许宿主机通过 RDMA 网卡（RDMA NIC，简称 RNIC）在没有远端节点参与的情况下直接发起对远端内存的读写请求，其数据传输方式如图14.9所示。这一特性绕过了远端服务器的系统内核，能直接访问远端用户态的地址空间，从而提供**高带宽**和**低时延**的优势。基于这一特性，学术界出现了不少网络应用的加速工作，如 Wukong [16] 利用 RDMA 来

加速对 RDF 的查询；FaSST [9] 使用 RDMA 优化过的 RPC 库来支持分布式事务处理等。

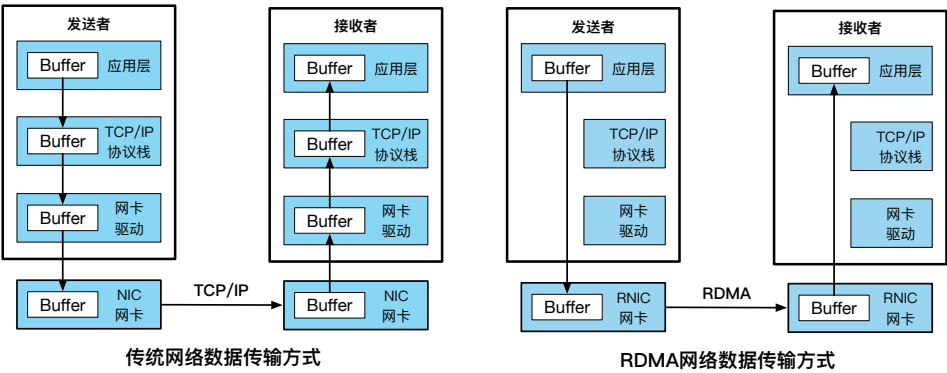


图 14.9: 传统网络数据传输方式（左）和基于 RDMA 网络的数据传输方式（右）对比。RDMA 方式绕过了双方的操作系统内核，因此具有更好的性能优势。

**Receive-side Scaling (RSS)：**为了利用处理器的多核算力，网卡引入了 RSS 技术，又称“多队列网卡”，允许将网卡上的不同接收队列分配给不同的 CPU 核心进行处理，从而共同分担网络收包的压力。例如 FlexNIC [10] 观察到数据中心众核的发展趋势，利用网卡 RSS 技术提升了网络应用吞吐能力在多核上的可扩展性。RSS 通常会结合一定哈希算法，做多核心间网络处理上的负载均衡。在实际应用中，系统还会将 RSS 限制在有限几个处理器核心上，避免网络中断给整体系统计算带来的影响。

**巨帧 (Jumbo Frames)：**巨帧可以将数据帧的长度从 1500 字节提升到 4074 字节甚至 9000 字节（不考虑尾部）。根据 TCP/IP 协议，数据帧头部的长度已经固定，巨帧可以让数据帧的有效载荷大大增加，通过更少的传输次数传输更多字节。目前巨帧主要用于数据中心内部的网络。由于互联网标准已经数据帧的**最大传输单元** (Maximum Transmission Unit, MTU) 规定为 1500 字节，从数据中心发往互联网的数据帧需要重新调整为 1500 字节。

**中断优化：**当中断的触发非常频繁时，处理器会疲于应付“中断风暴”的处理。中断优化的有效机制之一就是多个连续中断进行合并 (Interrupt Coalescing)，即等网络包积累到一定程度再产生一个中断，通知网卡驱动进行处理。之前介绍的 Linux NAPI 也可以视为中断优化的一种，触发中断后则转为轮询模式，从而降低整体中断的数量，避免后续中断对系统性能造成的影响。

**Scatter/Gather (SG)：**网卡设备的 RSS 特性和 Linux NAPI 都是针对收

包的优化，而 SG 则用于发包的优化。SG 本身也是系统里常见的优化技术，如 `writerv()` 和 `readv()` 系统调用。简单来讲，SG 允许数据从离散的多个区域分别读取，而不再是一块连续的内存空间。在网络优化中，SG 可以减少不必要的内存拷贝。例如，网卡驱动可以用一块内存作为数据包，另一块内存作为数据包的头部，然后将两部分内存都通知给网卡进行 SG。

**用户态网络：**对于传统的宏内核系统（如 Linux 发行版和 FreeBSD）而言，内核负责整个网络通信的命令控制和数据转发，所有的数据都需要通过内核来传递。而“用户态网络”优化的核心思想就是在网络的数据路径中彻底旁路内核（Bypass-Kernel）。如系统国际顶级学术会议 SOSP 1995 年的 U-Net [20]，将协议栈运行在用户态中，同时对网络接口进行虚拟化，允许多个应用通过多路复用接口直接访问网卡数据，实现了数据包在应用和网卡之间的零拷贝。类似的代表性工作还有来自韩国 KAIST 团队的 mTCP [8]、腾讯云团队的 F-Stack [1] 以及 Linux 基金会的 Vector Packet Processing (VPP) [2]。

### 14.6.2 系统设计：控制平面与数据平面分离

为了提升网络系统性能，系统设计者做了几类软件加速方案的探索。每种方案都有不同程度的效果，同时也带来了一定的代价：

- **弃用中断使用轮询处理高速网络数据：**为了降低高频中断带来的系统性能损耗，Linux 的 NAPI 模式在网卡的硬中断到来后，会在后续软中断上下文中使用轮询处理后续数据帧。类似的设计案例还有数据平面开发套件 (Intel DPDK)，我们将在下一小节对它更多的网络优化技术进行展开。
- **复用内核协议栈，但对套接字抽象进行修改：**MegaPipe [7] 发现基于虚拟文件系统 (VFS) 的套接字效率太低，因为在 Linux 中，套接字本身被当做文件描述符使用，因此需要初始化和文件系统相关的数据结构，如 `inode` 和 `dentry`。为此 MegaPipe 提出了一种轻量化的网络抽象 `lwsocket`，消除 VFS 抽象引入的开销，同时兼容网络套接字的语义。当然，这种舍却 VFS 的设计并非没有问题，在应用程序需要对套接字进行文件语义操作时，MegaPipe 必须将 `lwsocket` 转换为传统文件描述符，恢复 VFS 相关的数据结构。
- **弃用内核协议栈，使用用户态协议栈设计：**考虑到 Linux 内核态 TCP 短连接的效率低下，mTCP [8] 首先将 TCP 处理完全迁移到了用户态，将 `socket` 相关的系统调用变为函数调用，避免了模式开销和内存拷贝；

其次，单纯弃用中断使用轮询会消耗较多 CPU 时间周期，mTCP 则采用了 I/O 多路复用的思路，提出了针对网卡 TX/RX 队列的事件驱动接口 `ps_select()`；最后，mTCP 在每个核心上都部署了独立的 TCP 协议，核间不共享数据，从而实现多核上的可扩展性。需要注意的是，mTCP 的设计并不适合于长连接请求。

- **利用硬件虚拟化进行加速，把网络应用放在虚拟机的内核态：**库操作系统 OSv [11] 将网络协议栈视为 LibOS 的组成部分，在编译时根据需求和应用程序进行链接，在运行时同处于虚拟机的内核模式，因此套接字接口调用从系统调用转为了函数调用。此外，OSv 对网络协议栈进行了一定优化，为用户程序提供无锁、零拷贝的网络接口实现。相比于用户态协议栈设计，库操作系统网络架构还能利用硬件虚拟化的优势，例如 Arrakis [13] 使用 Single-Root I/O Virtualization (SR-IOV) 允许每台虚拟机都能独占地使用物理网卡，而不必下陷到虚拟机监控器，极大地提高了网络应用程序的性能。

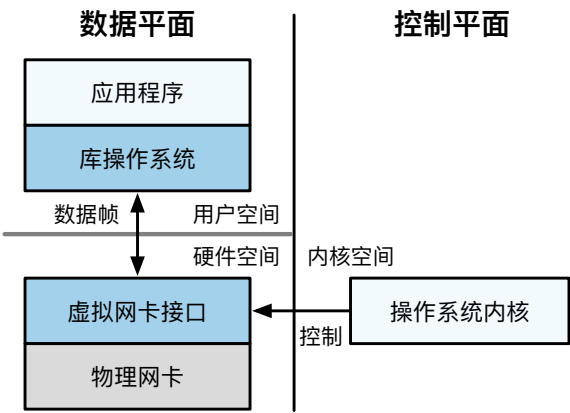


图 14.10: 控制平面与数据平面的示意图（库操作系统即 LibOS）

库操作系统 Arrakis[13] 的核心设计思想是将操作系统视为控制平面，和同年另一篇库操作系统 IX [5] 提出的将网络处理视为数据平面是非常类似的。这里就不得不谈到控制平面与数据平面分离的思想。受到用户态协议栈设计的 [18, 20] 的启发，以及当时较为火热的软件定义网络（Software Defined Network, SDN）思想，系统研究人员对数据中心的网络系统设计进行了重构，提出了控制平面与数据平面分离的加速思路。图14.10展示了网络系统中的控制平面和数据平面。从图中可以看到，操作系统负责充当控制平面，用于搭建

网卡到应用之间的连接通道；而网络数据包则在应用和网卡之间进行移动。通过这样的设计，将操作系统内核从网络处理的“快速路径”（Fast Path）中移除，使得数据中心内部的网络吞吐量大大提高 [5, 13]，甚至解决了 C10K 问题：即如何使单台机器能够支持 1 万个并发连接请求。

下文我们首先介绍工业界中广受关注的 Intel DPDK 套件，其中的 DP 就是数据平面的意思，它提供了一整套数据平面加速的软件方案。下一节我们介绍以智能网卡为代表的网络加速的硬件方案，它将原本机器上的网络处理过程卸载（offload）到了具备一定算力的新型网卡设备。

### 14.6.3 软件加速方案：Intel DPDK

随着网络设备性能越来越强劲，就连服务器端的 Linux 内核协议栈都出现了性能瓶颈。人们认真分析了 Linux 内核的特点，得出如下基本观察：

- **中断开销**：高频中断意味着高频模式切换。尽管 Linux 引入了软中断等优化机制，但软中断的频繁调度仍导致缓存一致性（cache coherence）、锁竞争（lock contention）等一系列压力。
- **内存拷贝**：数据包在内核空间被协议栈处理后，会将数据最终拷贝到进程对应的缓冲区内，数据包在不同空间下的拷贝也存在一定开销。
- **跨核处理**：一个数据包的生命周期可能横跨多个处理器核心<sup>1</sup>。这种跨核处理会导致大量缓存失效的问题。在 NUMA 架构下这个问题还会更为严重，CPU 跨 NUMA 节点远程访问内存将不可避免地增加时延。
- **TLB 缓存失效**：Linux 系统默认使用 4KB 的分页配置，由于 TLB 缓存空间有限，TLB 映射条目的频繁变更会产生大量的 TLB miss。

为了解决以上问题，Intel 联合其他厂商提出了一套全新的高效转发数据包的数据平面开发套件（Data Plane Development Kit），简称 DPDK [4]。不同于 Linux 内核协议栈，DPDK 是一套完全旁路内核（Bypass-Kernel）的网络设计，提供了用户态网络包的高效处理，方便开发各类高速转发的应用程序。

DPDK 充分利用了控制平面和数据平面分离的思想。首先网卡不再使用中断机制通知 CPU，因为中断本身也会产生一次模式切换，进入中断上下文；其次 DPDK 的数据处理旁路了内核协议栈，用户态程序可以直接访问网卡队

---

<sup>1</sup>例如，在处理器核心 0 上触发网络中断处理函数，在核心 1 上进行内核态的协议栈处理，在核心 2 上完成用户态（如 Web 服务器）的处理逻辑。

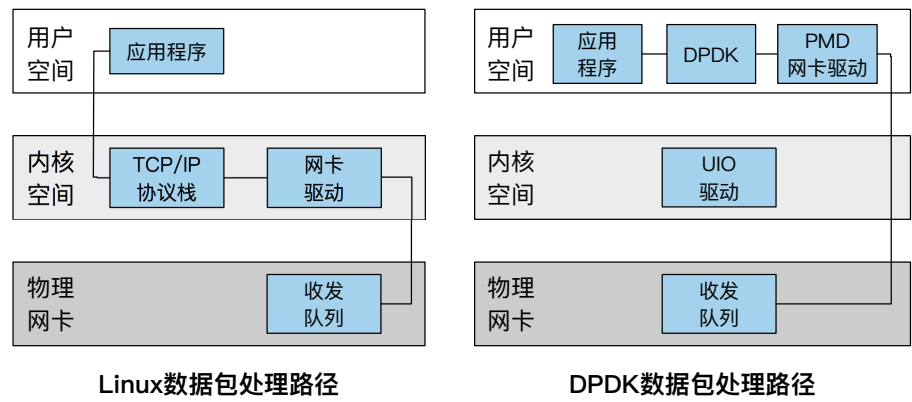


图 14.11: 传统 Linux 数据包处理路径（左）和 DPDK 数据包处理路径（右）

列上的数据包，实现了数据零拷贝。如下，我们介绍 DPDK 中性能优化的主要技术点。

用户空间驱动 (UIO)

UIO 是 Linux 提供给运行在用户空间的驱动代码访问设备 I/O 区间的机制。对于注册好的 UIO 设备，Linux 内核会对用户空间暴露 `/dev/uioX` 文件接口，对该文件的读写就是对设备内存的读写访问。借助 UIO 机制，DPDK 可以绕过内核协议栈直接和网卡交互，收发并处理网卡收发队列上的数据帧。除了 UIO 机制外，DPDK 还能通过 VFIO 实现用户态的 IO 驱动。VFIO 需要虚拟化的支持，借助 IOMMU 实现用户态的隔离。

轮询模式驱动

在传统的网络收包处理过程中，每当网卡设备收到一个新的数据帧后，都会向处理器发送一个中断。网卡中断的频繁请求会不断打断处理器的正常计算过程，导致网络应用性能的极大降低。为此，DPDK 设计了使用轮询模式的 PMD (Poll Mode Drivers) 网卡驱动。其思路和 NAPI 十分类似，即当第一次中断通知到来后切换为轮询模式收包。

任务绑核

操作系统在运行过程中可能对任务进行迁移。当任务跨核迁移时，原有核心上的缓存数据将被清除 (Cache Flush)。线程跨核调度和调度后的缓存缺失

都会导致新的开销。DPDK 借助 `pthread` 线程库将任务线程和 CPU 进行亲和性 (Affinity) 绑定, 避免被调度器跨核迁移造成的性能损失。值得注意的是, DPDK 提出了 **LCORE** 的概念, LCORE 对应的 `pthread` 绑定的 CPU 核是独占的, 无法和其他的 `pthread` 共享。而传统的 `pthread` 设置亲和性则允许多个 `pthread` 绑定到同一个 CPU 核上。

## 大页内存

操作系统默认使用 4KB 大小的标准页。当需要映射的表项非常多时, 会导致 TLB 缓存项的资源紧张, 产生频繁的 TLB miss。大页机制<sup>2</sup>允许一个 TLB 缓存项指向更大区域的 2MB 页面, 通过减少 TLB 项的使用来提高 TLB 的命中率。DPDK 使用 Linux 提供的“持久性大页” (Persistent Huge Pages), 持久性大页不会发生换页, 而是永远驻留在内存上, 因此不会导致内存访问时的缺页中断, 也能提高性能。DPDK 在大页的基础上实现了**内存池** (Mempool), 用于数据包的管理。内存池的内存分配是 NUMA 感知的, 避免跨节点内存访问带来的延迟影响。

## 无锁环形队列

DPDK 使用了无锁技术实现了无锁队列 `rte_ring`, 能够支持多写者/多读者模型。写者和读者间可以在不同核心上并行而不必加锁。无锁设计使用硬件原子指令和内存屏障来保证并行结果的正确性。对于队列的 `enqueue()` 和 `dequeue()` 操作上, DPDK 使用了批处理技巧, 即将多个对象用一个操作来完成。

### 14.6.4 硬件加速方案: 智能网卡🌶️🌶️

相比于传统网卡, 智能网卡集成了网络处理的加速单元和可编程单元。加速单元保证了网络处理的高性能, 而可编程单元提供了灵活的可编程性。这种设计允许把原来在宿主处理器 (Host CPU) 执行的计算逻辑卸载到智能网卡, 实现 **CPU - 智能网卡** 的异构协同计算。

考虑到实现成本、编程友好程度和灵活性的权衡, 智能网卡的实现方式大致分为三类。第一类是 **ASIC** 实现, 具有较强的专用芯片加速能力, 但加速逻辑基本固化在芯片电路中, 因此其上可支持的网络负载类型也会受到限制。第

---

<sup>2</sup>大页机制的具体介绍请参考本书第四章

二类是 **FPGA** 实现, **FPGA** 在性能方面和 **ASIC** 芯片非常接近, 但具备高度的可编程特性或灵活性。 **FPGA** 需要使用 Verilog/VHDL 等领域特定语言进行编程, 对应用程序开发者的编程要求也很高, 受限于 **FPGA** 的硬件逻辑资源, **FPGA** 智能网卡更适合于卸载并行性充分、局部性强的计算。第三类是 **SoC** 实现, 如 **ARM** 多核处理器, 此类实现具有如下特点: 1) **SoC** 通用处理器适合计算重复性不强, 但是数据结构和算法复杂度较高的应用; 2) 可以采用 **C/C++** 进行编程, 因此具备较好的编程友好性; 3) 支持标准 **DPDK** 或 **Linux** 软件栈, 因此兼容现有的软件生态。

智能网卡带来的好处主要体现在如下几个方面:

1. **节省宿主服务器时钟周期**: 将网络应用的部分计算从宿主处理器卸载到智能网卡后, 就能够把宿主服务器上的 **CPU** 时钟周期留给更需要的应用。例如, ClickNP [12] 将网络和存储虚拟化功能卸载到了智能网卡上, 进而为每台客户机节省 0.2 个 **CPU** 核。
2. **提升网络应用处理性能**: 卸载到智能网卡上的计算可能比在原宿主处理器上运行的效率更高, 因为智能网卡提供了高并行性的计算单元或定制化的加速单元, 或是通过减少或消除 **PCIe** 的传输提供了快速路径处理。例如, 微软 **Azure** 公有云把网络虚拟化功能卸载到智能网卡后, 虚拟机的网络吞吐量高达 31Gbps, 远超同期基于 **CPU** 实现网络虚拟化的谷歌云平台的 16Gbps 吞吐量 [6]。
3. **改善数据中心能源效率**: 智能网卡通过 **PCIe** 与宿主处理器进行通信却不共享内存, 这使得智能网卡和宿主处理器可以作为独立的计算机运行。由此智能网卡可用于构建分布式计算资源池, 进一步扩展数据中心的算力。由于智能网卡与宿主服务器共享电源和机箱, 采用智能网卡扩展算力有助于改善能源效率、降低经济成本。例如, Inc-OnDemand [19] 将应用卸载到 **FPGA** 智能网卡后, 证实智能网卡在取得性能改善的同时, 还可以取得 100 倍的能效改善。
4. **变革数据中心体系结构**: 工业界正在探索如何打破服务器的限制, 以资源为单位, 将数据中心构建为大型的计算和存储资源池。智能网卡可以绕过机器的参与, 实现数据中心的 **GPU**、**SSD**、**FPGA** 等资源的直接互连, 从而使以资源为单位重构数据中心体系结构的想法变为可能 [17]。

综上所述, 通过 **CPU-智能网卡** 混合架构实现协同计算, 可以达到节省机器时钟周期、改善服务性能和降低经济成本的目的。当前学术界和工业界都在

积极探索如何拓展智能网卡的全新应用场景，或者如何与现有网络系统进行更好的垂直整合。

## 14.7 思考题

1. 网络协议栈的分层设计有哪些优点？
2. 请用你自己的话来描述端到端观点，同时分别列举出三种适合和三种不适合使用这一观点的场景。
3. 在现有 TCP/IP 协议栈设计中，传输层的 TCP 协议和数据链路层都设计了一定的纠错机制。TCP 协议会对网络中的丢包进行了超时重传，数据链路层则对传输时发生错误的帧进行丢弃。请结合端到端观点，思考应该如何看待这种冗余。端到端观点是否依然适用于现代的网络系统设计？
4. 在 ChCore 的微内核模型下，网络包从网卡中断到应用程序收到总共经历了几次 IPC？请画出 IPC 路径图，并尝试设计一种优化，减少网络处理过程的 IPC。
5. 试分析宏内核的内核态协议栈、微内核的用户态协议栈，以及虚拟化场景下的库操作系统协议栈的优点和缺点？
6. 云计算中心需要搭建入侵检测系统 (IDS)，其本质是对网络包的包头和包身进行快速模式识别，及早发现入侵行为的发生。请设计一种方案，可以实现对数据包的检查，同时最小化检查导致的转发延迟？（提示：智能网卡、Intel DPDK、Linux BPF 等）

## 参考文献

- [1] F-stack is an user space network development kit with high performance based on dpdk, freebsd tcp/ip stack and coroutine api. <https://github.com/F-Stack/f-stack>. Accessed: 2021-02-05.
- [2] The vector packet processor (vpp). <https://fd.io/vppproject/vpptech/>. Accessed: 2021-02-05.
- [3] Inside the linux packet filter, part ii. <https://www.linuxjournal.com/article/5617>, 2002.
- [4] Learn how to get involved with dpdk. <https://www.dpdk.org/>, 2020.

- [5] Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. IX: A protected dataplane operating system for high throughput and low latency. In *11th USENIX Symposium on Operating Systems Design and Implementation, OSDI '14, Broomfield, CO, USA, October 6-8, 2014*, pages 49–65. USENIX Association, 2014.
- [6] Daniel Firestone, Andrew Putnam, Sambrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat, Vivek Bhanu, Adrian M. Caulfield, Eric S. Chung, Harish Kumar Chandrappa, Somesh Chaturmohta, Matt Humphrey, Jack Lavier, Norman Lam, Fengfen Liu, Kalin Ovtcharov, Jitu Padhye, Gautham Popuri, Shachar Raindel, Tejas Sapre, Mark Shaw, Gabriel Silva, Madhan Sivakumar, Nisheeth Srivastava, Anshuman Verma, Qasim Zuhair, Deepak Bansal, Doug Burger, Kushagra Vaid, David A. Maltz, and Albert G. Greenberg. Azure accelerated networking: Smartnics in the public cloud. In *15th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2018, Renton, WA, USA, April 9-11, 2018*, pages 51–66. USENIX Association, 2018.
- [7] Sangjin Han, Scott Marshall, Byung-Gon Chun, and Sylvia Ratnasamy. Megapipe: A new programming interface for scalable network I/O. In *10th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2012, Hollywood, CA, USA, October 8-10, 2012*, pages 135–148. USENIX Association, 2012.
- [8] Eunyoung Jeong, Shinae Woo, Muhammad Asim Jamshed, Haewon Jeong, Sunghwan Ihm, Dongsu Han, and KyoungSoo Park. mtcp: a highly scalable user-level TCP stack for multicore systems. In *Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2014, Seattle, WA, USA, April 2-4, 2014*, pages 489–502. USENIX Association, 2014.
- [9] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Fasst: Fast, scalable and simple distributed transactions with two-sided (RDMA) data-gram rpcs. In *12th USENIX Symposium on Operating Systems Design*

*and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*, pages 185–201. USENIX Association, 2016.

- [10] Antoine Kaufmann, Simon Peter, Naveen Kr. Sharma, Thomas E. Anderson, and Arvind Krishnamurthy. High performance packet processing with flexnic. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '16, Atlanta, GA, USA, April 2-6, 2016*, pages 67–81. ACM, 2016.
- [11] Avi Kivity, Dor Laor, Glauber Costa, Pekka Enberg, Nadav Har'El, Don Marti, and Vlad Zolotarov. Osv - optimizing the operating system for virtual machines. In *2014 USENIX Annual Technical Conference, USENIX ATC '14, Philadelphia, PA, USA, June 19-20, 2014*, pages 61–72. USENIX Association, 2014.
- [12] Bojie Li, Kun Tan, Layong Larry Luo, Yanqing Peng, Renqian Luo, Ningyi Xu, Yongqiang Xiong, and Peng Cheng. Clicknp: Highly flexible and high-performance network processing with reconfigurable hardware. In *Proceedings of the ACM SIGCOMM 2016 Conference, Florianopolis, Brazil, August 22-26, 2016*, pages 1–14. ACM, 2016.
- [13] Simon Peter, Jialin Li, Irene Zhang, Dan R. K. Ports, Doug Woos, Arvind Krishnamurthy, Thomas E. Anderson, and Timothy Roscoe. Arrakis: The operating system is the control plane. In *11th USENIX Symposium on Operating Systems Design and Implementation, OSDI '14, Broomfield, CO, USA, October 6-8, 2014*, pages 1–16. USENIX Association, 2014.
- [14] Jerome H. Saltzer, David P. Reed, and David D. Clark. End-to-end arguments in system design. *ACM Trans. Comput. Syst.*, 2(4):277–288, 1984.
- [15] Savannah. lwip - a lightweight tcp/ip stack. <https://savannah.nongnu.org/projects/lwip/>, 2019.
- [16] Jiaxin Shi, Youyang Yao, Rong Chen, Haibo Chen, and Feifei Li. Fast and concurrent RDF queries with rdma-based distributed graph exploration. In *12th USENIX Symposium on Operating Systems Design and*

- Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*, pages 317–332. USENIX Association, 2016.
- [17] Ran Shu, Peng Cheng, Guo Chen, Zhiyuan Guo, Lei Qu, Yongqiang Xiong, Derek Chiou, and Thomas Moscibroda. Direct universal access: Making data center resources available to FPGA. In *16th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2019, Boston, MA, February 26-28, 2019*, pages 127–140. USENIX Association, 2019.
- [18] Chandramohan A. Thekkath, Thu D. Nguyen, Evelyn Moy, and Edward D. Lazowska. Implementing network protocols at user level. In *Proceedings of the ACM SIGCOMM '93 Conference on Communications Architectures, Protocols and Applications, San Francisco, CA, USA, September 13-17, 1993*, pages 64–73. ACM, 1993.
- [19] Yuta Tokusashi, Huynh Tu Dang, Fernando Pedone, Robert Soulé, and Noa Zilberman. The case for in-network computing on demand. In *Proceedings of the Fourteenth EuroSys Conference 2019, Dresden, Germany, March 25-28, 2019*. ACM, 2019.
- [20] Thorsten von Eicken, Anindya Basu, Vineet Buch, and Werner Vogels. U-net: A user-level network interface for parallel and distributed computing. In *Proceedings of the Fifteenth ACM Symposium on Operating System Principles, SOSP 1995, Copper Mountain Resort, Colorado, USA, December 3-6, 1995*, pages 40–53. ACM, 1995.

网络协议栈与系统：扫码反馈



